

Crash Consistency: FSCK and Journaling

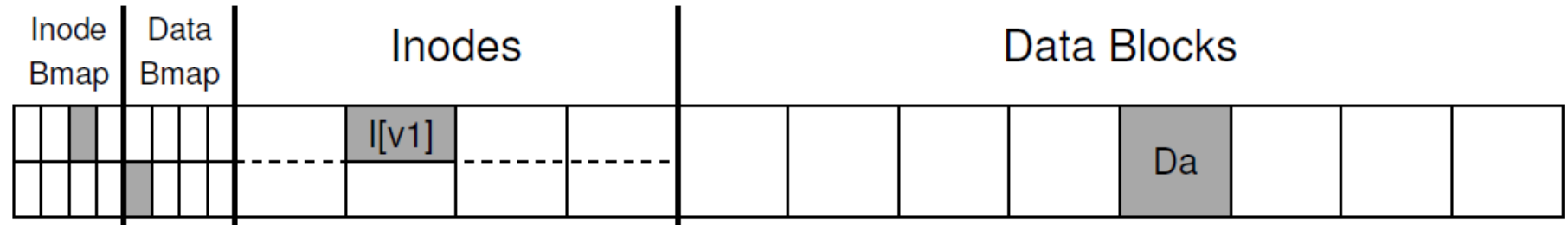
Chapter 42 - OSTEP

What happens if the system crashes?

- The OS maintains different data structures on disk
- A disk operation updates multiple of them
- These data structures must be persistent and consistent
- What if in the middle of a disk operation:
 - There's a power loss?
 - The OS encounters a bug and crashes?
- The data structures must be consistent across failures. This is called the **crash-consistency problem**.

Example

- HD state v1



- Inode data:

```
owner      : remzi
permissions : read-write
size       : 1
pointer    : 4
pointer    : null
pointer    : null
pointer    : null
```

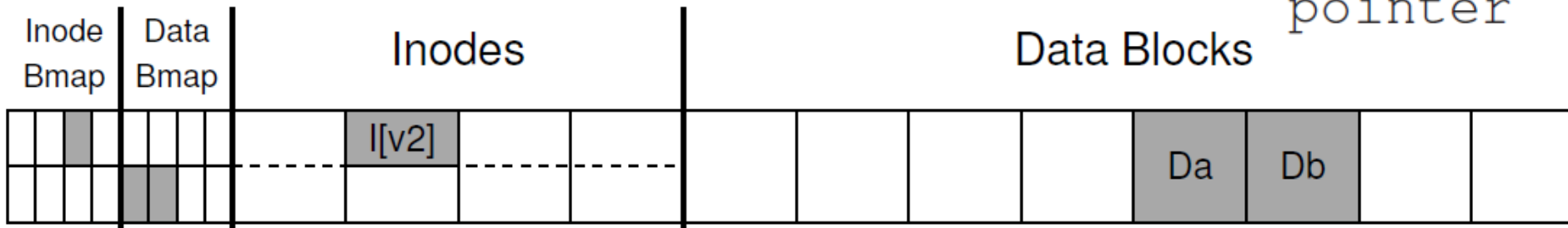
Example contd.

Append a block (4k) to his file.

- This requires updating 3 on-disk structures:
 - d-bitmap (to allocate the new block)
 - d-block (to write the new data in the appended block)
 - file-inode (to associate the block with this file)

- After update HD state v2 →

```
owner      : remzi
permissions : read-write
size       : 2
pointer    : 4
pointer    : 5
pointer    : null
pointer    : null
```



Crash scenarios:

- Just the data block (Db) is written to disk.
- Just the updated inode (I[v2]) is written to disk.
- Just the updated bitmap (B[v2]) is written to disk.
- The inode (I[v2]) and bitmap (B[v2]) are written to disk but not the data block (Db).
- The inode (I[v2]) and the data block (Db) are written to disk but not bitmap (B[v2]).
- The the data block (Db) and bitmap (B[v2]) are written to disk but not inode (I[v2]).

File System Checker (FSCK)

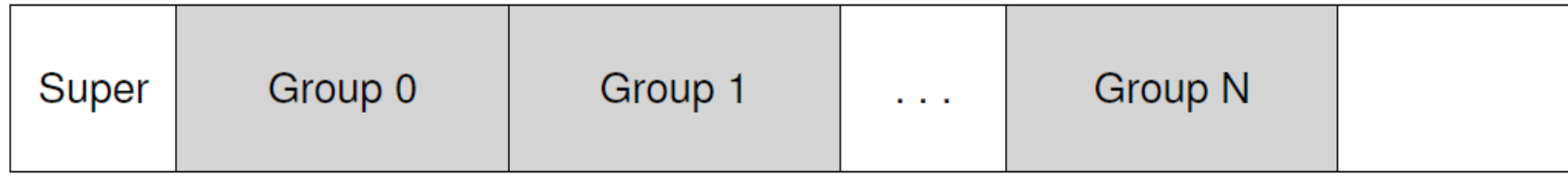
- Earlier systems let inconsistencies happen and tried to fix them on the next boot.
- fsck is on such tool on unix which checks:
 - Superblock
 - Free blocks
 - Inode state
 - Inode links
 - Duplicate blocks
 - Bad block pointers
 - Directory checks
- Issues:
 - Slow: Scans the whole disk and disks are becoming bigger and bigger
 - Expensive: Have to scan the whole disk even to detect a small inconsistency

Journaling (Write-Ahead Logging)

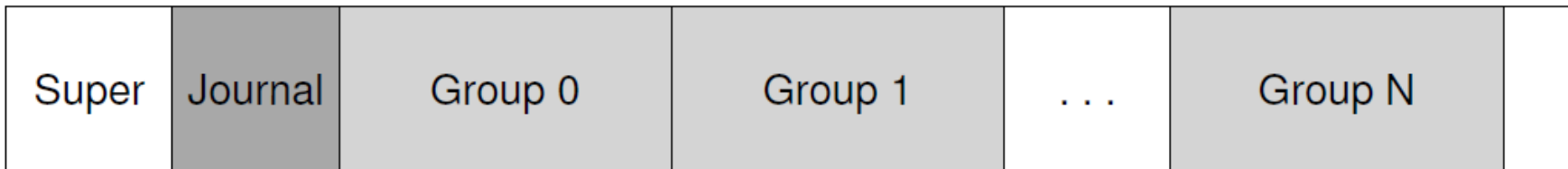
- **Idea:** when performing a disk operation, before updating the structures, first “write ahead” *what you are going to do*, in a “log”
- If a crash happens during the operation, you can go back, look at the note you wrote in log, and try again: you will know exactly what to fix, in stead of scanning the whole hard disk
- Journaling adds a bit of overhead (logging) in writes, but saves lots of time in recovery

Example

- Linux ext2 file system (without journaling):

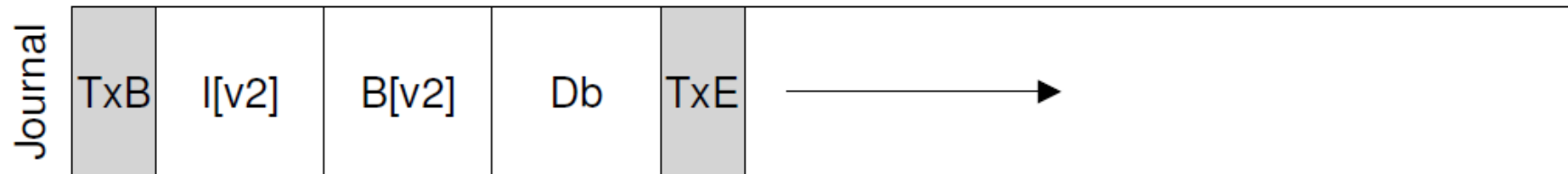


- Linux ext3 file system



Example: Data Journaling

- Before doing your update, write your intentions in the log first:



- TxB (transaction begin): Info about the update (addresses of I[v2], B[v2], Db), TID, etc
- Middle blocks: contents of blocks themselves (physical logging)
- TxE (transaction end): Marks end of update, also contains TID

Write happens in two steps

After the 2nd step we are done.

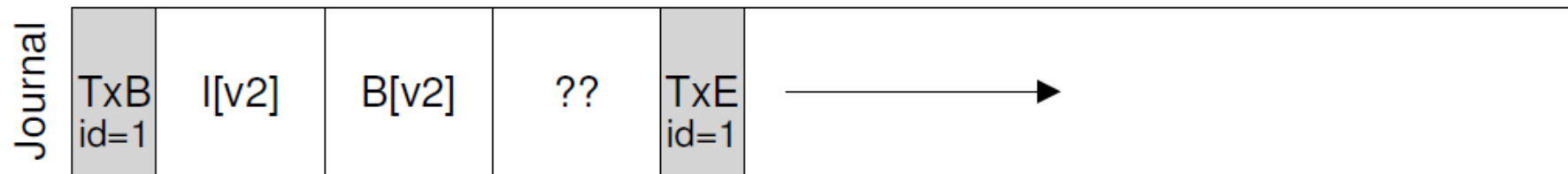
1. **Journal write:** Write the transaction, including a transaction-begin block, all pending data and metadata updates, and a transaction-end block, to the log; wait for these writes to complete.
2. **Checkpoint:** Write the pending metadata and data updates to their final locations in the file system.

We first write all the info+data to the journal, then to the disk.

What if the crash happens before the write to journal is completed?

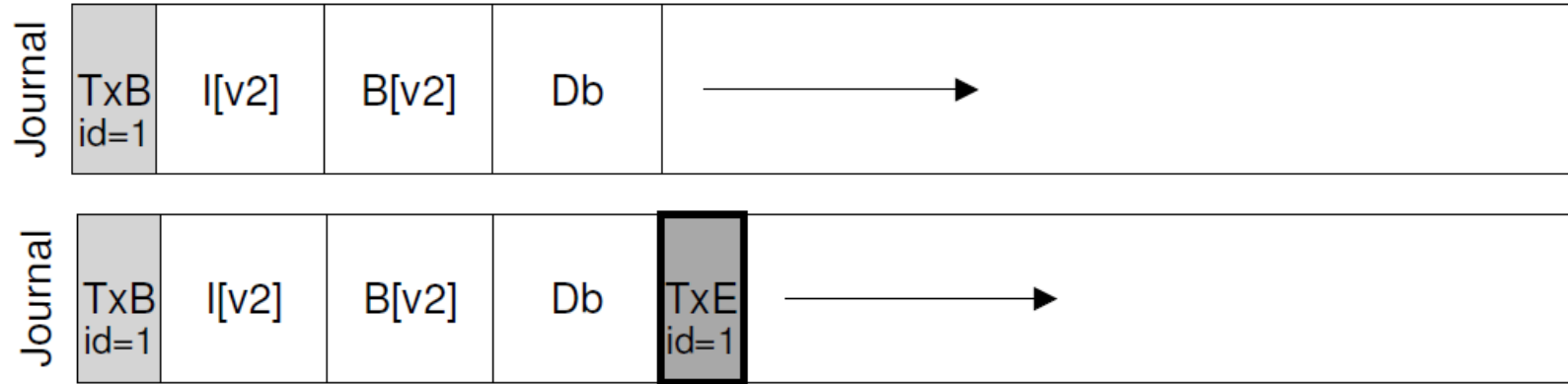
Journal write

- If we issue all the writes to journal (e.g., TxB, I[v2], B[v2], Db, TxE) in parallel (desirable for performance) or as a single big write, the disk might break it down and reorder them for optimization. A crash in between might leave the transaction seemingly correct but wrong:



Here the crash happened before Db was written. Looks like valid transaction (has TxB and TxE) but Db location has garbage. Writing it to disk will corrupt data.

Journal write (in two steps)



TxE is 512 bytes to make sure its atomic.

1. **Journal write:** Write the contents of the transaction (including TxB, metadata, and data) to the log; wait for these writes to complete.
2. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for write to complete; transaction is said to be **committed**.
3. **Checkpoint:** Write the contents of the update (metadata and data) to their final on-disk locations.

Recovery From a Crash

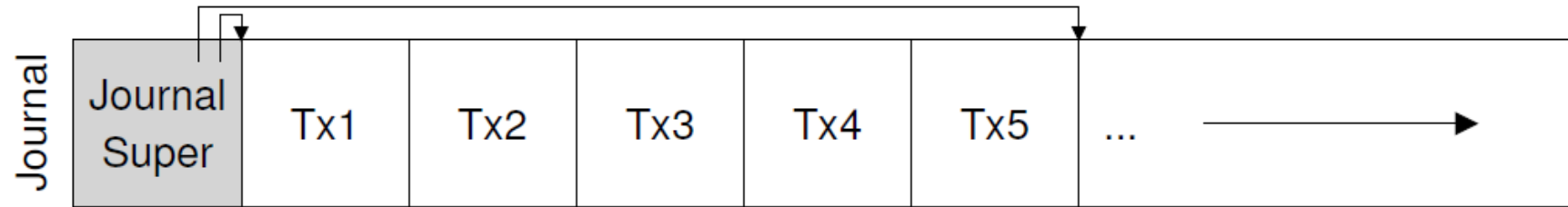
- A crash may happen anytime during these steps:
 1. Before step 1 completed:
 - In recovery we simply skip the incomplete transaction
 2. After step 1 (the transaction was committed to the log):
 - At boot time, the FS scans the log, and replays (in order) the committed transactions (Redo logging)
 3. After step 2, in step 3:
 - The incomplete checkpoint will be replayed and data will be rewritten (some overhead of rewriting blocks what were already written)

Batching updates

- Writing each write separately to disk creates lot of traffic
- E.g. Create two files in the same directory:
 - Same inode bitmap updated twice
 - Their inodes may be in same directory
 - Their entries will be made in the same data block of directory
- Some systems buffer the updates in memory (mark the in-memory inode bitmap, inode table, etc dirty) into a global transaction, and write the big transaction once after some timeout.

Maintaining the log

- Log has finite space, we have to free up completed updates. One way is to maintain a circular list of pending transactions:



Journaling steps

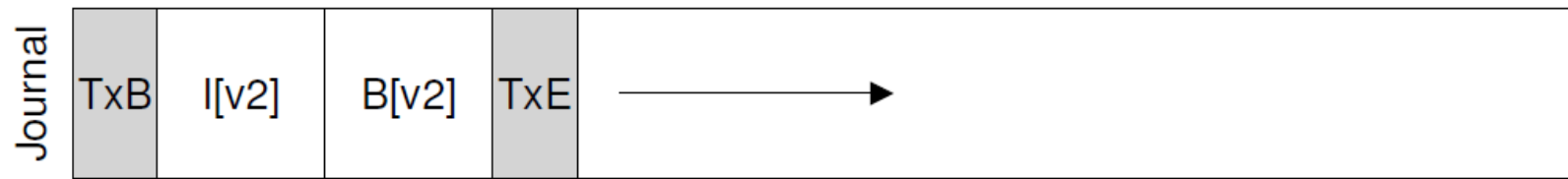
1. **Journal write:** Write the contents of the transaction (containing TxB and the contents of the update) to the log; wait for these writes to complete.
2. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for the write to complete; the transaction is now **committed**.
3. **Checkpoint:** Write the contents of the update to their final locations within the file system.
4. **Free:** Some time later, mark the transaction free in the journal by updating the journal superblock.

STILL: we are writing each block to disk twice: once to the journal, once to the disk!

Metadata journaling (Ordered journaling)

Avoiding double block writes

- Only metadata is journaled (not user data)



- Db is written directly to the disk
- But when? Writing it after can produce consistent system with garbage data in case of a crash

Metadata journaling: order of writes

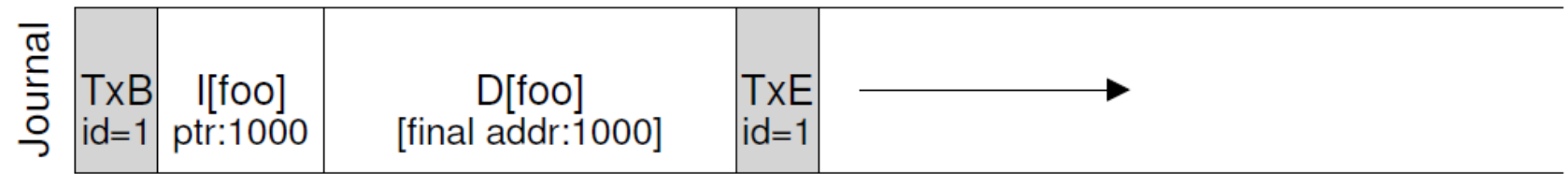
1. **Data write:** Write data to final location; wait for completion (the wait is optional; see below for details).
2. **Journal metadata write:** Write the begin block and metadata to the log; wait for writes to complete.
3. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for the write to complete; the transaction (including data) is now **committed**.
4. **Checkpoint metadata:** Write the contents of the metadata update to their final locations within the file system.
5. **Free:** Later, mark the transaction free in journal superblock.

Block reuse problem in delete

- In case of crash a replay will corrupt file data

- Ext3 revokes deletions
- Revoked records are not played in recovery.

The particular example Tweedie gives is as follows. Suppose you are using some form of metadata journaling (and thus data blocks for files are *not* journaled). Let's say you have a directory called `foo`. The user adds an entry to `foo` (say by creating a file), and thus the contents of `foo` (because directories are considered metadata) are written to the log; assume the location of the `foo` directory data is block 1000. The log thus contains something like this:



At this point, the user deletes everything in the directory and the directory itself, freeing up block 1000 for reuse. Finally, the user creates a new file (say `foobar`), which ends up reusing the same block (1000) that used to belong to `foo`. The inode of `foobar` is committed to disk, as is its data; note, however, because metadata journaling is in use, only the inode of `foobar` is committed to the journal; the newly-written data in block 1000 in the file `foobar` is *not* journaled.

